APLICAÇÃO DESKTOP EM .NET

# Nex_TI Desktop

Uma aplicação desktop pequena e organizada: flashcards com revisão espaçada.

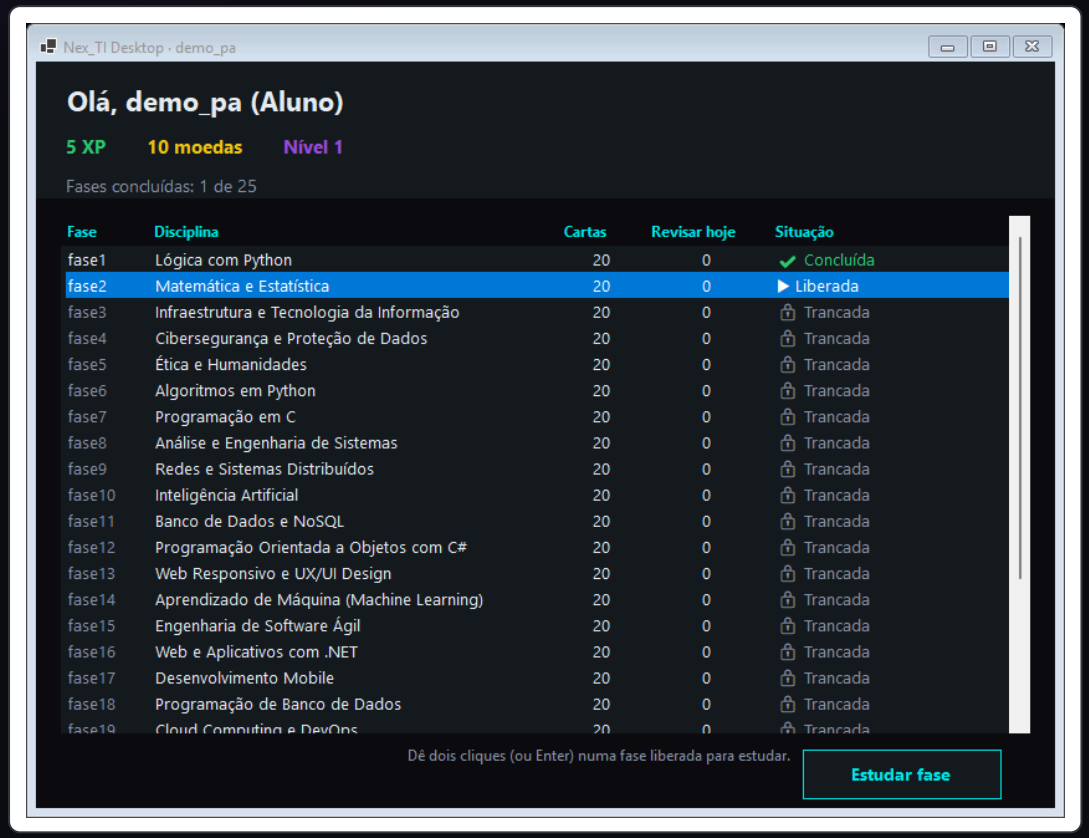
**DevTime**

Gabriel Alves Moreira (H67HJI4) · Maciel Costa da Silva (R280985) · Maycon Douglas Inácio Silva (H719CD3)

Miguel Angel Fernandez Ortiz (H7858F9) · Rafael Mesquita (H6722I0)

github.com/MayconDIS/PIM_IV · sistema/apps/desktop

# Estudar TI no computador, sem internet




### Entrar

O aluno digita só o nickname. Os dados ficam no computador dele.

### Trilha

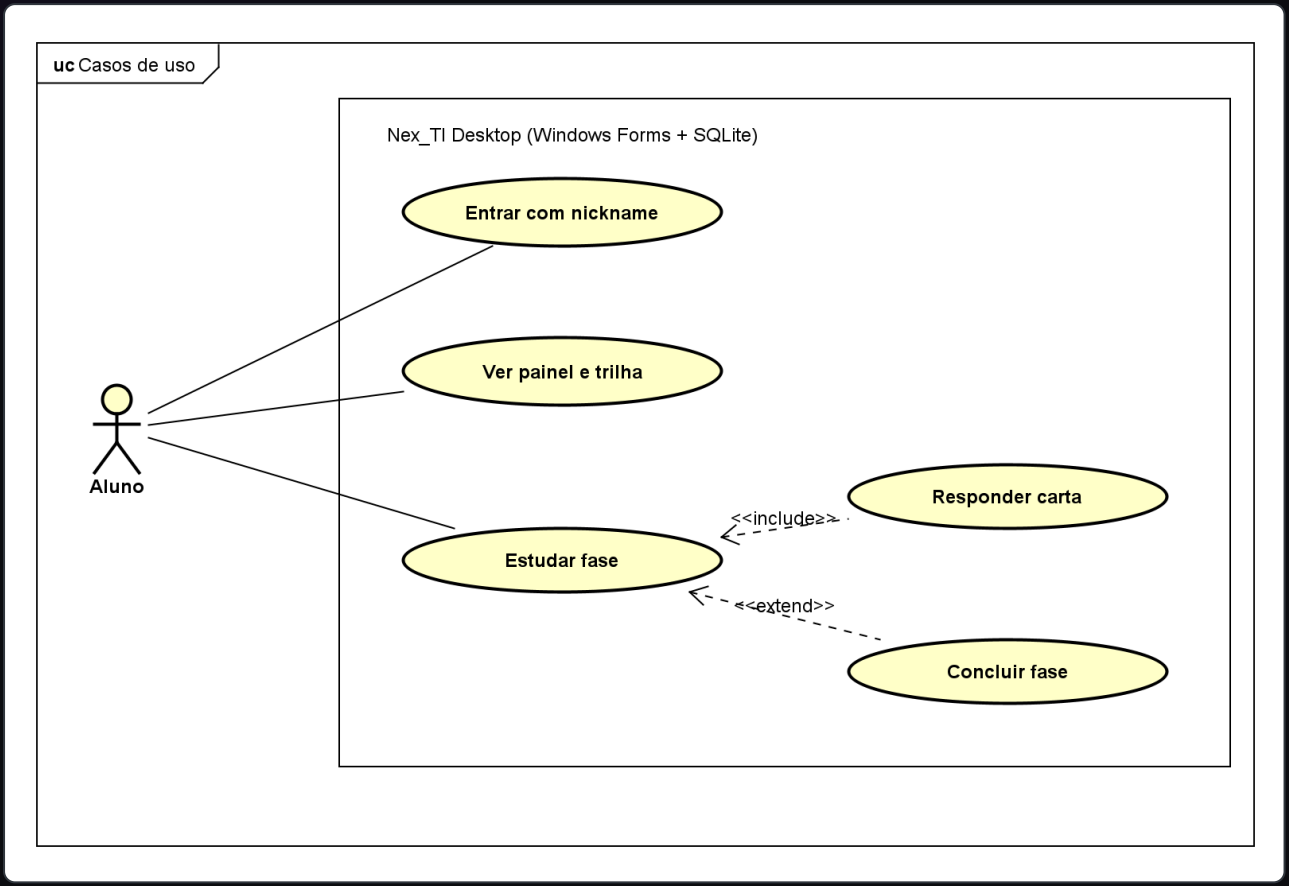
25 fases e 582 cartas sobre TI. Concluir uma fase libera a próxima.

### SM-2, XP e moedas

O SM-2 decide quando cada carta volta. A fase dá XP e moedas só na primeira vez.

**Windows Forms** + **.NET 10** + **SQLite**.

# O que o aluno faz no programa



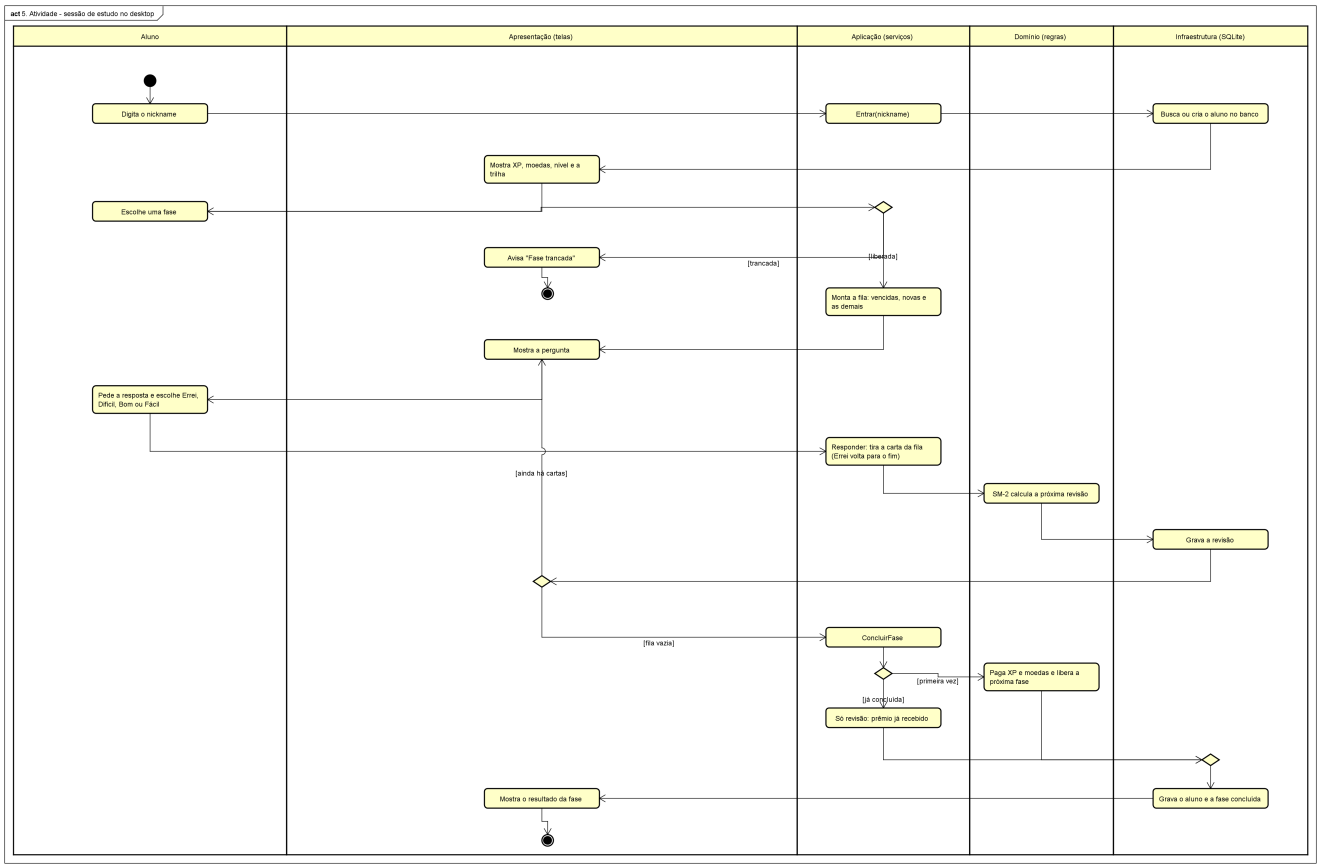

**Ator:** o Aluno.

**Entrar com nickname**, **ver painel e trilha** e **estudar fase**.

**«include»:** estudar uma fase sempre inclui responder cartas.

**«extend»:** concluir a fase só acontece quando a última carta é acertada.

# Uma sessão de estudo, passo a passo




1. O aluno entra; o programa busca ou cria o aluno no banco.
2. Fase trancada? Aparece um aviso.
3. Liberada: a fila começa pelas cartas vencidas.
4. Cada resposta passa pelo SM-2 e é gravada.
5. Fila vazia: paga XP e moedas (só na primeira vez).

Uma raia para o aluno e uma para cada camada.

# Onde a POO aparece no Nex_TI Desktop

### Herança

O Aluno é um Usuario: herda o Id, o nickname e a data de criação.

```
class Aluno : Usuario
```

### Abstração e polimorfismo

Usuario é abstrata; cada tipo diz o próprio perfil.

```
public abstract string Perfil { get; }
```

### Encapsulamento

XP e moedas só mudam por ReceberPremio, que paga uma vez por fase.

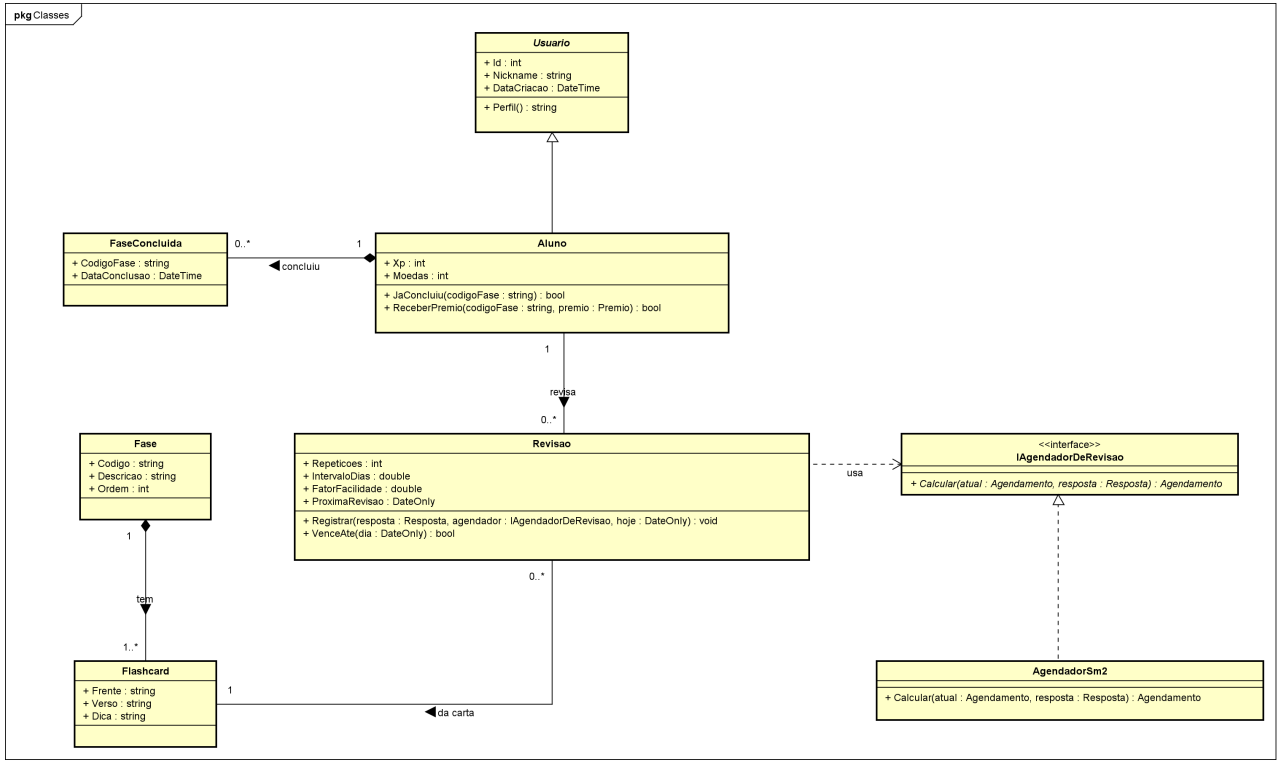
```
public int Xp { get; private set; }
```

### Interface

A revisão usa o contrato; o SM-2 é quem faz. Dá para trocar o algoritmo.

```
class AgendadorSm2 : IAgendadorDeRevisao
```

# As classes do domínio




**Herança:** Aluno herda de Usuario.

**Composição:** o Aluno é dono das fases concluídas; a Fase, das cartas.

**Revisao** guarda o SM-2 de cada carta para cada aluno.

**Interface:** IAgendadorDeRevisao, implementada pelo AgendadorSm2.

# Cada camada cuida de uma coisa

### Apresentação

As telas em Windows Forms: entrada, painel e estudo.

`Telas/`

### Aplicação

Os casos de uso: entrar, listar fases, estudar e concluir.

`ServicoDeEstudo`

### Domínio

Entidades, a regra do SM-2 e as interfaces dos repositórios.

`Entidades/ e Regras/`

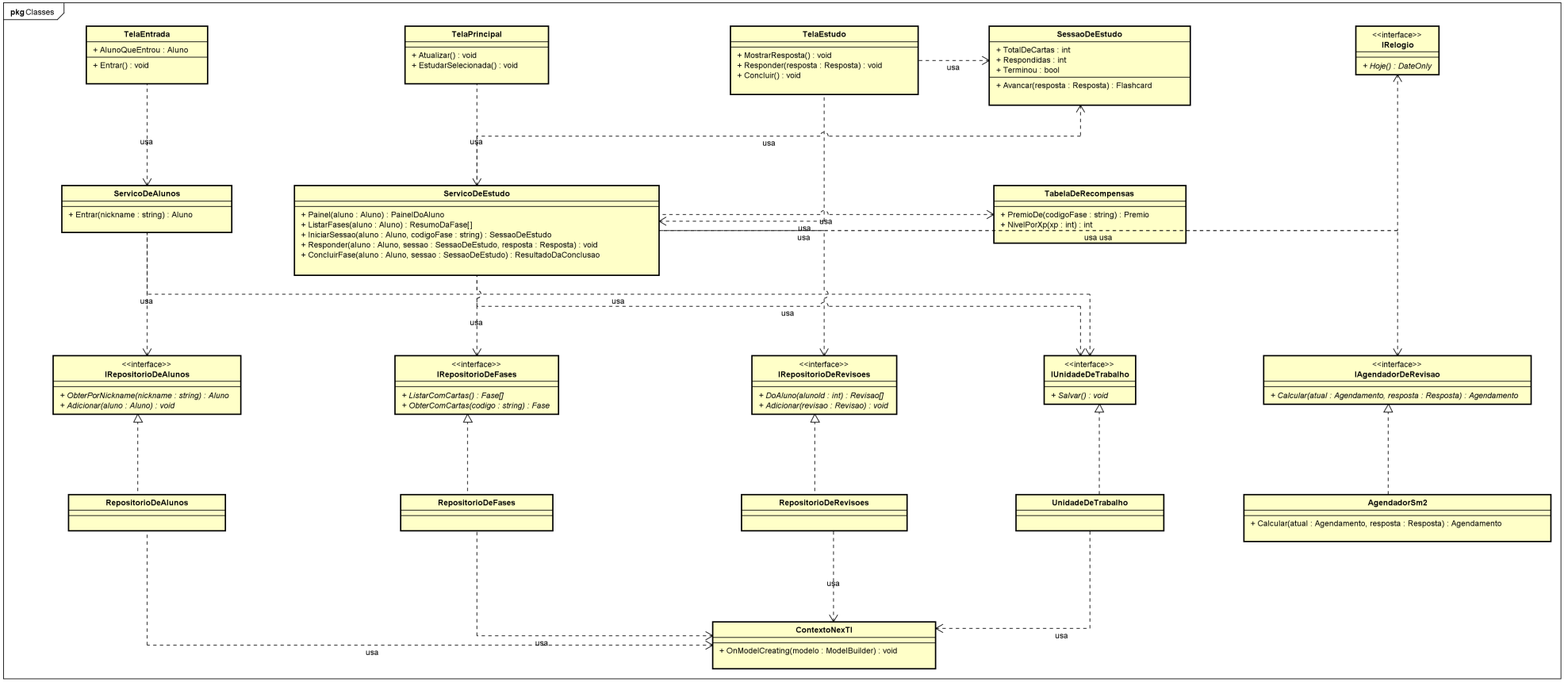
### Infraestrutura

O banco SQLite pelo EF Core: contexto, repositórios e migration.

`ContextoNexTI`

**Regra:** cada camada só usa a de baixo. O **Domínio não depende de ninguém**, e a Aplicação conhece só as interfaces, sem saber que existe SQLite.

# As camadas no diagrama de classes

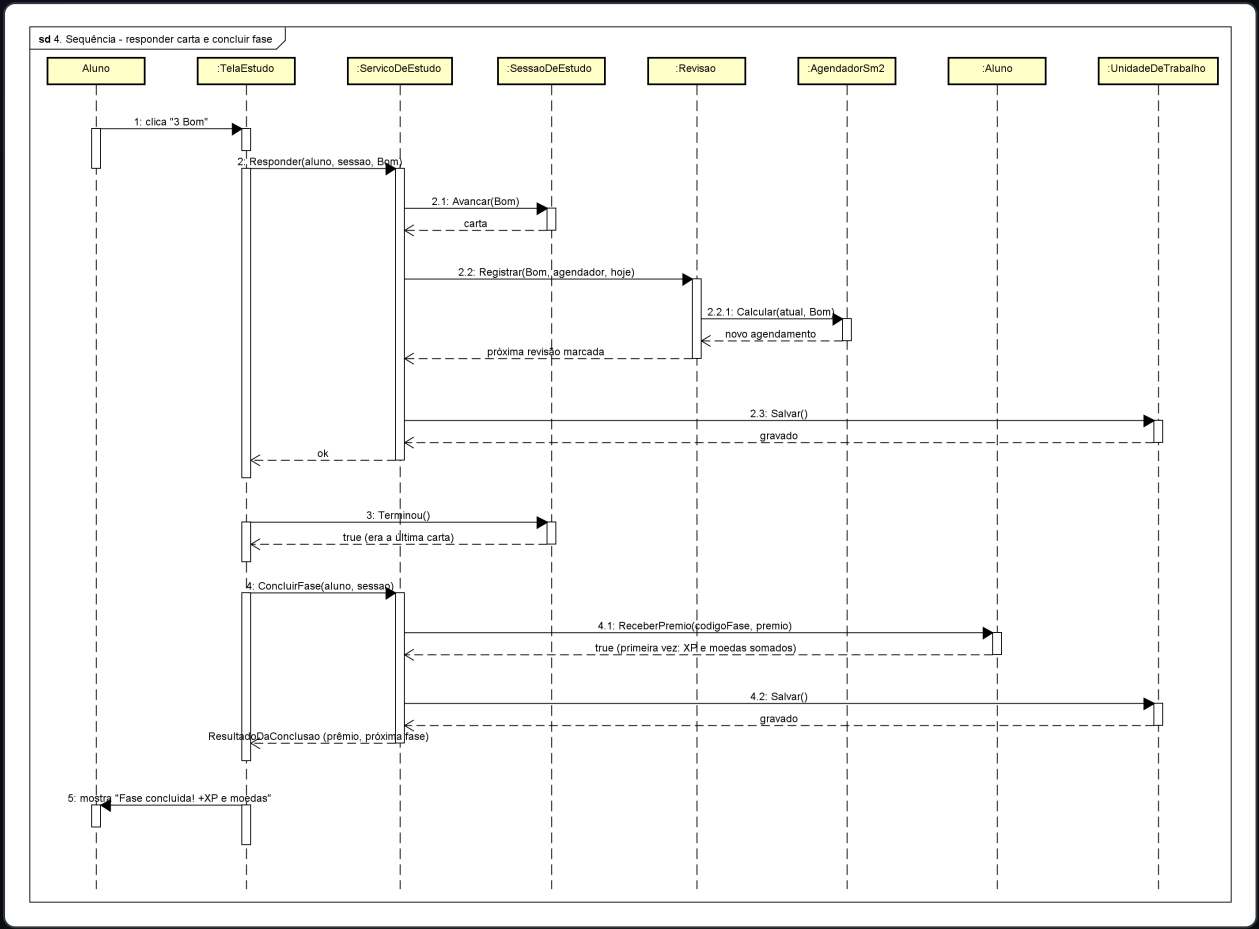


# Cada camada é um projeto separado

| PROJETO | O QUE TEM | USA |
|---|---|---|
| `NexTI.Desktop.Apresentacao` | As telas e o Program.cs, que liga as camadas | Aplicação e Infraestrutura |
| `NexTI.Desktop.Aplicacao` | ServicoDeAlunos, ServicoDeEstudo e SessaoDeEstudo | Domínio |
| `NexTI.Desktop.Dominio` | Entidades, SM-2 e interfaces | nenhum |
| `NexTI.Desktop.Infraestrutura` | EF Core + SQLite: contexto, repositórios, migration e semente | Domínio |
| `NexTI.Desktop.Testes` | 27 testes automáticos (xUnit) | Domínio, Aplicação e Infraestrutura |

**Por quê:** o compilador não deixa o Domínio usar a tela ou o banco. Os testes trocam o banco por um em memória sem mudar as regras.

# Responder a última carta e concluir a fase




A tela chama **Responder** no serviço.

O serviço tira a carta da fila, o **AgendadorSm2** calcula a próxima revisão e a **UnidadeDeTrabalho** grava.

Era a última carta? **ConcluirFase**: o Aluno recebe XP e moedas se for a primeira vez.

`ServicoDeEstudo.Responder()`

# As telas e o banco







### EF Core + SQLite

Um arquivo de banco no computador do aluno, sem servidor.

### Migration e semente

A migration Inicial cria as 5 tabelas; a semente coloca as fases e as cartas.

### Atalhos

Espaço mostra a resposta; 1 a 4 respondem Errei, Difícil, Bom e Fácil.

# Testes e GitHub

**27**
testes automáticos

**4**
camadas, uma por projeto

**5**
tabelas no SQLite

**4**
tipos de diagrama UML

Pull Request → GitHub Actions roda os testes → Merge na main

**Os testes** conferem o SM-2, o prêmio uma vez só e o programa de ponta a ponta, com um banco SQLite em memória.

# Obrigado!

Perguntas?

`github.com/MayconDIS/PIM_IV` · `sistema/apps/desktop`